CSED233 · 데이터구조 · 포스텍

Ch10. AVL Tree

1. 왜 Balanced Tree인가?

문제 상황

- 일반 BST의 worst case = $O(n)$ (skewed tree)
- 삽입 순서가 나쁘면 (예: 1,2,3,4,5 순차) 높이가 $n-1$
- **보장된 $O(\log n)$이 필요** → height-balanced tree

복잡도 비교

자료구조	Worst	Average
일반 BST(이진 탐색 트리)	$O(n)$	$O(\log n)$
Balanced BST (AVL)	$O(\log n)$	$O(\log n)$

- AVL은 **Search·Insert·Delete·Min/Max의 worst-case에 $O(\log n)$ 보장**, 단 전체 노드를 방문하는 Inorder traversal은 O(n)

2. Height-Balanced 정의

두 가지 수준의 balance

> **Perfectly balanced**: 모든 level이 꽉 참 (= 모든 내부 노드 자식 2개 + 모든 leaf가 같은 level; 같은 level만으로는 full이 아님). 높이가 h이면 노드 수가 정확히 2^{h+1}-1이어야 하므로, 노드 한 개 삽입만으로 perfect 상태를 유지하는 것은 일반적으로 불가능하다. 그래서 완화된 height-balanced 조건을 쓴다.

> **(Almost) Height-Balanced**: 각 노드 x에 대해 **두 subtree의 높이 차 ≤ 1**, 그리고 두 subtree도 재귀적으로 height-balanced → **AVL tree 조건**.

Height 규약

- **단일 노드**: height = 0
- **빈 tree**: height = -1 (non-existing subtree)
- → Balance factor 계산 시 일관성 유지

Proof (sketch): almost balanced → height $O(\log n)$

- $N(h)$ = 높이 h AVL tree의 **최소** 노드 수
- 점화식: $N(h) = N(h-1) + N(h-2) + 1$ (한쪽 $h-1$, 다른쪽 $h-2$)
- 귀납으로 **$N(h) \geq \phi^h$** ($h \geq 0$): 기저는 h = 0에서 **$N(0) = 1 = \phi^0$**, h = 1에서 **$N(1) = 2 \geq \phi$**이며, 귀납 단계($h \geq 2$)에서는 $\phi^{h-1} + \phi^{h-2} = \phi^h$ ($\phi^2 = \phi + 1$ 이용)이므로 $N(h) = N(h-1) + N(h-2) + 1 \geq \phi^h + 1 \geq \phi^h$ — Fibonacci 수열과 유사, ϕ는 황금비 ≈ 1.618
- 따라서 $n \geq \phi^h \rightarrow$ **$h \leq \log_\phi n = O(\log n)$** ✓ (점화식 초기값: **$N(-1) = 0$, $N(0) = 1$**이라 $N(1) = 2$이며, $N(-1) = 0$은 점화식용 초기값일 뿐 부등식의 귀납 기저가 아니다)

3. Balance Factor(균형 인수)

정의

$$bf(x) = \text{height}(x.\text{left}) - \text{height}(x.\text{right})$$

- **AVL property**: $-1 \leq bf(x) \leq 1$ for every node x
- bf 값:
  - $+1$: left-heavy
  - 0: balanced
  - -1: right-heavy
  - ± 2: **violation** → rotation 필요

4. Insertion: 기본 절차

2단계

1. **BST insert**: 일반 BST 방식으로 leaf에 추가
2. **Rebalance**: 새 노드에서 root 방향으로 올라가며 **balance factor 체크**, 위반 시 **rotation**

핵심 관찰

- **Insert 후 balance 변화는 경로상의 노드만** (leaf → root 경로)
- **첫 번째 violation 노드 = Pivot (A)** → 가장 깊은 unbalanced node
- 이 pivot에서만 rotation하면 **전체 복구 보장** (증명: rotation 후 subtree 높이가 insert 이전과 같아짐)

5. Imbalance 4 Types

4가지 경우 (pivot = A)

Type	의미	사례
LL	A의 left child의 **left** subtree에 삽입	left-left 편향
RR	A의 right child의 **right** subtree에 삽입	right-right 편향
LR	A의 left child의 **right** subtree에 삽입	zig-zag
RL	A의 right child의 **left** subtree에 삽입	zig-zag

Rotation(회전) 종류

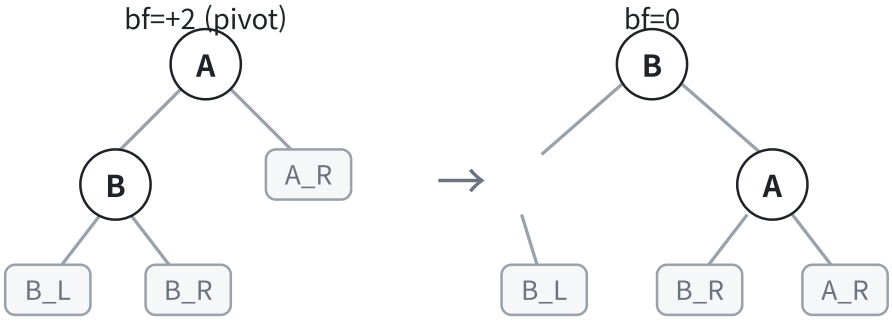
- **Single rotation**: LL, RR (pivot 한 번만 회전)
- **Double rotation**: LR, RL (두 번: 먼저 child 회전, 다음 pivot 회전)
  - **LR = RR + LL**: child에 RR 먼저, 그 다음 pivot에 LL
  - **RL = LL + RR**: child에 LL 먼저, 그 다음 pivot에 RR

왜 LR, RL은 double rotation?

- LR 형태를 single LL rotation만 하면 여전히 불균형
- 먼저 중간 child에 RR로 돌려 → LR을 'LL 형태로 변환' → 그 다음 LL

6. Single Rotation: LL Case

절차 (Right Rotation)



1. B_R이 A의 왼쪽 자식이 됨
2. B가 A의 자리를 대체
3. A가 B의 오른쪽 자식이 됨

Right Rotation(LL): B_R만 주인이 바뀌고, inorder 순서는 B_L B B_R A A_R로 불변 (회전 전·후 동일 - B_R만 A의 왼쪽 subtree로 이동)

핵심 조작

- B (pivot의 left child)가 새 root
- 원래 A는 B의 right child로
- B의 원래 right subtree (B_R)는 새 A의 left subtree로

시간

- $O(1)$ (pointer 조작 몇 개만)
- 전체 insert: BST search $O(\log n)$ + 재균형은 최대 한 위치에서 수행 (기본 회전은 LL/RR 1번, LR/RL 2번) = $O(\log n)$

7. Double Rotation: LR Case

상황

- Pivot(피벗) A의 left child B의 right subtree에 새 노드 삽입 → LR 타입

Step 1: B를 축으로 Left (RR) rotation

- → 이제 **LL 타입으로 변환됨**

Step 2: A를 축으로 Right (LL) rotation

- 결과: C가 새 root, B와 A가 각각 left/right child
- **LR = RR + LL** ✓

RL Case (symmetric): RL = LL + RR

8. Insertion 시간복잡도

분석

Step	시간
BST insert	$O(\log n)$
Rebalance path trace	$O(\log n)$
각 노드 bf 계산	$O(1)$
Rotation (재균형 최대 한 위치; 기본 회전 LL/RR 1번·LR/RL 2번)	$O(1)$
Total	$O(\log n)$

전제: 각 노드에 subtree height를 저장한다고 가정하면 height 갱신과 BF 계산은 노드당 O(1)이고, 경로상 유지 비용 전체는 O(log n)이다. 저장된 높이가 없으면 노드당 O(1)이 성립하지 않는다.

Early termination

- **재균형은 최대 한 위치**에서만 수행 (기본 회전은 LL/RR 1번, LR/RL 2번)

- 재균형 후 해당 subtree 높이가 **insert 이전 높이로 복귀**
- → 그 위 조상들의 bf 변화 없음 → 즉시 종료 가능 (early termination)

9. Deletion

절차

1. **BST deletion** 수행 (leaf/deg-1/deg-2 case 처리 — Ch9)
2. **실제 삭제된 노드 위치에서 root 방향으로** bf 체크
3. Violation 발생하면 **적절한 rotation**
4. **Insertion과 달리, 여러 번 rotation이 필요할 수 있음** (계속 위로 올라가며 체크)

어떤 rotation을 선택할까?

Pivot x에서 위반 감지 시:

- B = x의 **더 높은 쪽 child**
- C = B의 **더 높은 쪽 child**
- A-B-C 경로의 L/R 패턴으로 type 결정 (LL, LR, RR, RL)

B의 자식 높이가 같을 때

중요 규칙: B의 두 자식 높이가 같다면 **single rotation 선택** (LL or RR)

- 이유: B의 두 자식 높이가 같으면 더 높은 자식 C를 유일하게 정할 수 없어 **single rotation을 선택하는 것이 표준 규칙**이다. single rotation을 쓰면 AVL 조건이 복구되고 subtree 높이가 삭제 이전과 같아져 조상 검사를 종료할 수 있다.
- → '동률이면 single 우선'

Deletion 복잡도

- 각 rotation $O(1)$, 경로 길이 $O(\log n)$
- **Insertion과 달리 rotation 최대 $O(\log n)$번** 발생 가능
- Total = $O(\log n)$

Q1. 왜 insert는 rotation 1번, delete는 여러 번?

힌트

회전이 서브트리의 "높이"를 어떻게 바꾸는지가 핵심이다.

정답

개념은 힌트 참고.

- Insert 후 rotation: subtree 높이가 insert 이전과 **동일**해짐 → 조상 영향 없음
- Delete 후 rotation: subtree 높이가 1 **감소**할 수 있음 → 조상도 unbalanced 될 수 있음 → 계속 올라가며 체크

Q2. 일반 BST가 AVL보다 실전에서 빠른 경우?

힌트

AVL의 bf 관리와 회전은 상수 비용이 크다. 균형이 이미 보장된다면?

정답

개념은 힌트 참고.

- 무작위 순열로 삽입하면 기대 높이와 기대 탐색은 O(log n)이지만 AVL 균형 조건을 보장하지 않으며 worst-case는 여전히 O(n)이다. 입력이 **이미 random**이면 일반 BST도 $O(\log n)$, rotation 오버헤드 없음
- AVL은 매 삽입/삭제마다 bf 관리 + rotation → 상수항 큼. worst-case 보장이 중요하지 않고 실제 높이가 낮다면, 회전·BF 유지 비용이 없는 일반 BST가 빠를 수 있다
- → 'worst case 보장이 필요 없는' 경우 일반 BST도 실용적

Summary

연산	AVL Tree
Search	$O(\log n)$
Insert	$O(\log n)$, 재균형 최대 한 위치 (기본 회전 LL/RR 1번·LR/RL 2번)
Delete	$O(\log n)$, 최대 $O(\log n)$ **rotations**

Min/Max	$O(\log n)$
Inorder (sorted)	$O(n)$

핵심:

- AVL property: $|bf(x)| \leq 1$
- 4 imbalance types: **LL, RR, LR, RL**
- Single rotation (LL/RR) vs Double rotation (LR = RR+LL, RL = LL+RR)
- Insert: pivot에서만 재균형 (기본 회전 LL/RR 1번·LR/RL 2번) → 조상 자동 복구
- Delete: 조상까지 연쇄 rotation 필요 가능
- Height 보장: Fibonacci-bound에 의해 $h = O(\log n)$